A2C Framework

Architecture-to-Code

A Developer Platform Framework for Generating Enterprise-Grade Code, IaC, and CI/CD Using Agentic AI

Built on: E2A Architecture Framework (github.com/subhamviky/e2a-framework)

Author: Subham Gupta, Staff Architect & AI Architect

A2C extends E2A. E2A governs agentic AI systems. A2C uses those governed agents to generate enterprise-grade software.

Core thesis: The same architectural discipline that makes enterprise systems reliable should be enforced at code generation time, not discovered through production failures.

> **The Meta-Insight**
> E2A governs how agentic AI systems are built. A2C applies E2A's own governance mechanism (BaseAgent.run() with idempotency, latency SLO, policy enforcement, quality gates) to GENERATE code that ITSELF follows E2A patterns. The agent is governed. The output is governed. The framework is self-referential.

1. Executive Summary

The Architecture-to-Code (A2C) Framework is a developer platform methodology that uses E2A-governed agentic AI to generate production-grade software artifacts: microservice code, Infrastructure-as-Code (IaC), CI/CD pipelines, and NFR implementations. It addresses a fundamental gap: code generators produce functional code but not architecturally sound code. A2C enforces enterprise architecture patterns — idempotency, circuit breakers, observability, policy-as-code — at generation time through agent governance, not through post-generation review.

A2C extends E2A in one precise direction: E2A defines how to build governed agentic AI systems. A2C uses those governed agents as the engine for a developer platform that generates governed software. The developer specifies the requirement. The agent enforces the architecture.

Dimension	E2A Framework	A2C Framework
Purpose	Build governed agentic AI applications	Use governed agents to generate enterprise software
Primary abstraction	BaseAgent, BaseRAGPipeline, BaseToolService	SDLCAssistantAgent, CriticAgent, CodeGenTool, IaCGenTool, CICDGenTool
Public entry point	agent.run(state, config)	sdlc_agent.run(dev_request, config) — same pattern
Output	Agent responses, tool results, RAG answers	Python/Java microservice code, Terraform IaC, GitHub Actions workflows, NFR implementations
NFR enforcement	Inherited by all agent subclasses from BaseAgent	Injected into all generated code artifacts via _apply_policy()

2. The Problem A2C Solves

2.1 Code Generation Without Architecture

Current LLM-based code generation tools (GitHub Copilot, Cursor, Claude artifacts) produce syntactically correct code that fails the production-readiness test. The failure modes are consistent across tools and teams:

Failure Mode	Description	Production Consequence
NFR Amnesia	Generated code has no idempotency, no circuit breakers, no retry logic, no structured logging. These are not in the prompt.	Duplicate writes, cascading failures, silent errors, undebuggable production incidents.
Architecture Drift	LLM generates business logic directly in route handlers, skipping service layers, interface contracts, and data models.	Untestable components, tight coupling, impossible to maintain at scale.
Governance Gap	No approval gates, no policy-as-code, no PII redaction, no tool allow/deny lists in generated code.	Compliance failures discovered in production, not at generation time.
IaC Afterthought	Infrastructure is manually configured in the cloud console, not version-controlled or reproducible.	Environment drift, deployment failures, no audit trail, impossible to recreate.
CI/CD Bolted On	Pipelines are added after the code works, not designed as quality gates from day one.	No RAGAS gate, no vulnerability scan, no auto-rollback. Production is the test environment.

2.2 What A2C Fixes

A2C treats NFR enforcement, IaC, and CI/CD not as post-generation additions but as mandatory generation targets. The SDLCAssistantAgent's _apply_policy() method programmatically injects required NFRs into the generation contract before the LLM call. The agent cannot produce a microservice that lacks idempotency or observability — not because the developer remembered to ask, but because the architecture enforces it structurally.

> **The A2C Design Principle**
> Architecture is not a prompt. It is a constraint. A2C embeds the architectural constraint (NFR requirements, Clean Architecture layers, E2A patterns) into the agent's governance layer (_apply_policy) before any LLM call. The developer specifies WHAT to build. A2C determines HOW it must be built. The output is production-ready by construction, not by review.

3. A2C Framework Architecture

3.1 The Meta-Abstraction

A2C is a meta-application of E2A. The same abstract class hierarchy that governs agentic AI systems is used to build the developer platform that generates those systems. This is not coincidence — it is the correct structural choice. Every A2C agent inherits NFR enforcement from E2A's BaseAgent. Every generated artifact follows the E2A pattern because the generating agent enforces it through its governance layer.

Layer	E2A (governs agents)	A2C (generates governed code)
Data model	AgentState TypedDict	DevRequest TypedDict (project_name, project_type, mandatory_nfrs, context, target_cloud)
Interface contract	BaseAgent abstract class	SDLCBaseAgent extends BaseAgent (same contract, SDLC domain)
Business logic	Agent class: LLM + tool calls	SDLCAssistantAgent: LLM generates code + IaC + CI/CD artifacts
Quality gate	CriticAgent: RAGAS groundedness	CodeCriticAgent: validates generated code contains mandatory NFR patterns
Tool layer	BaseToolService: HTTP microservice calls	CodeGenTool (writes files), IaCGenTool (validates Terraform), CICDGenTool (generates YAML)
Orchestration	BaseWorkflow: LangGraph StateGraph	SDLCWorkflow: routes DevRequest through Code → IaC → CICD → Critic agents

3.2 Multi-Agent SDLC Workflow

A production A2C implementation is a multi-agent LangGraph workflow. A single agent handling code + IaC + CI/CD in one LLM call is a prototype pattern. The correct architecture separates concerns across specialised agents, each governed by E2A's BaseAgent contract.

SDLCWorkflow — LangGraph StateGraph topology

```
# SDLCWorkflow routes DevRequest through 5 specialist agents:
# Each agent inherits run() from BaseAgent. LLM call via config['model_id'].

DevRequest (input)
    ↓
RequirementsAgent        ← validates project_type, mandatory_nfrs, target_cloud
    ↓
CodeGenAgent             ← generates Clean Architecture microservice code
    ↓
IaCAgent                 ← generates Terraform / Azure Bicep / CDK IaC
    ↓
CICDAgent                ← generates GitHub Actions workflows with RAGAS gate, Trivy, rollback
    ↓
CodeCriticAgent          ← validates all artifacts: NFRs present, architecture layers correct
    ↓
DeliveryState (output)   ← project structure + all files + validation report
```

3.3 NFR Injection Pattern

The _apply_policy() hook in each agent is the enforcement point. It runs before the LLM call and modifies the DevRequest state to guarantee mandatory NFRs are included in the generation contract. This is the key architectural decision: governance enforced structurally, not as a prompt suggestion.

NFR injection sequence in _apply_policy()

```
# _apply_policy() runs at Step 3 in BaseAgent.run() — before LLM call
MANDATORY_NFRS = [
    'Idempotency',           # __ensure_idempotency + DynamoDB conditional write
```

```
    'Observability',          # structlog JSON + OTel tracing + CloudWatch metrics
    'CircuitBreaker',         # pybreaker per downstream service
    'RetryWithBackoff',       # tenacity decorator with jitter
    'HealthCheck',            # /health endpoint for ECS/K8s readiness probe
    'GracefulShutdown',       # SIGTERM handler for zero-downtime deploys
]

def _apply_policy(self, state: DevRequest, config=None, **kwargs) -> None:
    # 1. Inject missing mandatory NFRs into generation contract
    injected = []
    for nfr in MANDATORY_NFRS:
        if nfr not in state.get('mandatory_nfrs', []):
            state['mandatory_nfrs'].append(nfr)
            injected.append(nfr)
    if injected:
        logging.info(f'Policy injection: added mandatory NFRs: {injected}')
    # 2. Validate target cloud is in approved list
    approved = (config or {}).get('approved_clouds',
                ['aws','gcp','azure','standalone'])
    if state.get('target_cloud','').lower() not in approved:
        raise GovernanceViolationError(
            f"Cloud {state['target_cloud']} not in approved list")
    # 3. Set idempotency key for this generation request
    # (BaseAgent.__ensure_idempotency handles request_id; this is the
    # business-level dedup key for the project itself)
    state['generation_key'] = (
        f"{state['project_name']}:{state['project_type']}"
```

3.4 CodeCriticAgent

CodeCriticAgent — evaluates generated code quality

```
# CodeCriticAgent uses a second LLM call to evaluate generated artifacts
# Returns float [0.0-1.0] based on architectural completeness
class CodeCriticAgent(BaseAgent):
    agent_name = 'CodeCriticAgent'

    def _validate_state(self, state, config=None, **kwargs) -> bool:
        return 'generated_code' in state and 'mandatory_nfrs' in state

    def _build_messages(self, state, config=None, **kwargs) -> list:
        nfrs = ', '.join(state['mandatory_nfrs'])
        return [{'role':'system','content':
            'You are a senior architect. Score 0.0-1.0: does this code '
            'contain all required NFR patterns? Return JSON: {score, missing, passed}'},
          {'role':'user','content':
            f'Required NFRs: {nfrs}\n\nCode:\n{state["generated_code"]}'}]

    def _apply_policy(self, state, config=None, **kwargs) -> None:
        # Use cheaper model for evaluation (cost-optimisation)
        config['model_id'] = config.get(
            'critic_model_id', 'anthropic.claude-3-haiku-20240307-v1:0')

    def _evaluate_output(self, response, state, config=None, **kwargs) -> float:
        # Parse structured score from critic LLM
        import json
        try:
            result = json.loads(response.get('text','{}'))
            score = float(result.get('score', 0.0))
            state['critic_report'] = result
            return score
        except (json.JSONDecodeError, ValueError):
            return 0.0
```

```
    def _fallback(self, state, config=None, **kwargs) -> str:
        return 'Code review failed. Regenerating with stricter NFR prompt.'

    def _handle_error(self, error, state, config=None, **kwargs):
        logging.error(f'CodeCriticAgent: {error}')
        state.setdefault('errors',[]).append(str(error))
        return state
```

4. SDLCAssistantAgent

DevRequest TypedDict — structured input replacing plain dict

```
from typing import TypedDict, List, Optional

class DevRequest(TypedDict, total=False):
    project_name:   str          # e.g. 'SettlementEngine'
    project_type:   str          # 'FastAPI Microservice' | 'Spring Boot' | 'LangGraph Agent'
    mandatory_nfrs: List[str]    # NFRs developer requested (A2C will add missing mandatory ones)
    context:        str          # free-text domain context
    target_cloud:   str          # 'aws' | 'gcp' | 'azure' | 'standalone'
    language:       str          # 'python' | 'java'
    # Written by A2C agents:
    generation_key: str          # business idempotency key
    generated_code: str          # output from CodeGenAgent
    generated_iac:  str          # output from IaCAgent
    generated_cicd: str          # output from CICDAgent
    critic_report:  dict         # output from CodeCriticAgent
    trace_id:       str          # written by BaseAgent.run()
    request_id:     str          # written by BaseAgent.__ensure_idempotency
    errors:         List[str]    # error accumulator
```

SDLCAssistantAgent

```
# SDLCAssistantAgent — generates enterprise-grade microservice code
# Inherits run() from BaseAgent. All NFR enforcement inherited.
# __llm_call routes to provider via config['model_id'] — not overridden.
from src.e2a_base import BaseAgent, GovernanceViolationError
import logging, os

MANDATORY_NFRS = ['Idempotency','Observability','CircuitBreaker',
                  'RetryWithBackoff','HealthCheck','GracefulShutdown']

class SDLCAssistantAgent(BaseAgent):
    agent_name = 'SDLCAssistantAgent'

    def _validate_state(self, state, config=None, **kwargs) -> bool:
        # Required: project context must be fully specified
        required = ['project_name','project_type','mandatory_nfrs','target_cloud']
        if not all(k in state for k in required):
            logging.error('Missing required project fields')
            return False
        return True

    def _apply_policy(self, state, config=None, **kwargs) -> None:
        # 1. Inject missing mandatory NFRs
        injected = [n for n in MANDATORY_NFRS
                    if n not in state.get('mandatory_nfrs', [])]
        state['mandatory_nfrs'].extend(injected)
        if injected:
            logging.info(f'A2C policy injection: {injected}')
        # 2. Validate approved cloud
```

```
        approved = (config or {}).get('approved_clouds',
                   ['aws','gcp','azure','standalone'])
        if state.get('target_cloud','').lower() not in approved:
            raise GovernanceViolationError(
                f"Cloud {state['target_cloud']} not approved")
        # 3. Business dedup key (NOT idempotency — that is BaseAgent.__ensure_idempotency)
        state['generation_key'] = (
            f"{state['project_name']}:{state['project_type']}"
            f":{state['target_cloud']}")

    def _build_messages(self, state, config=None, **kwargs) -> list:
        # Construct generation prompt encoding E2A patterns as requirements
        nfr_list = ', '.join(state['mandatory_nfrs'])
        lang = state.get('language', 'python')
        cloud = state.get('target_cloud', 'aws')

        system_prompt = (
            f'You are a Principal Cloud Architect generating production-ready {lang} code. '
            f'Generate a {state["project_type"]} named {state["project_name"]} for {cloud}. '
            f'Mandatory architecture requirements:\n'
            f'1. Clean Architecture: separate data model, interface contract, '
            f'   service layer, exposure layer.\n'
            f'2. All NFRs structurally implemented: {nfr_list}.\n'
            f'3. NFRs must appear as code (decorators, middleware, classes) '
            f'   not as comments.\n'
            f'4. Include structured logging (JSON), OTel tracing, /health endpoint.\n'
            f'5. Follow E2A Template Method: one public entry point, '
            f'   protected domain methods, private infrastructure helpers.'
        )
        return [
            {'role':'system','content': system_prompt},
            {'role':'user',  'content':
             f'Project context: {state.get("context","standard microservice")}'}
        ]

    def _evaluate_output(self, response, state, config=None, **kwargs) -> float:
        # Structural check: verify critical NFR keywords present in generated code
        # Production: delegate to CodeCriticAgent for LLM-based evaluation
        text = str(response).lower()
        required_patterns = {
            'idempotency':   any(k in text for k in
                             ['idempotent','request_id','conditional write']),
            'observability': any(k in text for k in
                             ['logging','structlog','opentelemetry','otel']),
            'circuit_break': any(k in text for k in
                             ['circuit','breaker','pybreaker','resilience']),
            'health_check':  any(k in text for k in ['/health','healthcheck']),
        }
        passed = sum(required_patterns.values())
        score = passed / len(required_patterns)
        state['nfr_check_detail'] = required_patterns
        return score

    def _fallback(self, state, config=None, **kwargs) -> str:
        # Corrected: fallback to E2A framework templates, not fabricated URL
        missing = [k for k,v in
                   state.get('nfr_check_detail',{}).items() if not v]
        return (f'Code quality gate failed. Missing NFR patterns: {missing}. '
                f'Refer to: github.com/subhamviky/e2a-framework/docs/ '
                f'for E2A implementation templates.')

    def _handle_error(self, error, state, config=None, **kwargs):
        # Structured error logging with project context
        logging.error(f'[SDLCAssistantAgent]'
                      f'[{state.get("project_name","?")}] {error}')
        state.setdefault('errors',[]).append(str(error))
        return state
```

```
# run() is NOT overridden. Inherits from BaseAgent:
# idempotency, latency SLO, token budget, observability log, trace_id.

# Usage:
agent = SDLCAssistantAgent(
    config={
        'model_id': 'anthropic.claude-3-5-sonnet-20241022-v2:0',
        'approved_agents': ['SDLCAssistantAgent'],
        'approved_clouds': ['aws','gcp','azure'],
        'min_confidence': 0.75,    # 3 of 4 NFR patterns must be present
        'max_latency': 30.0,       # code gen takes longer than agent calls
        'max_tokens': 32000        # code gen requires large token budget
    },
    state=dev_request
)
result = await agent.run(dev_request, agent.config)
```

5. NFR-First Generation Sequence

A2C applies the same NFR-First development sequence from the E2A Enterprise Framework to code generation. Every generated project follows this sequence — the artifacts are produced in the order that reflects production-readiness priority.

Phase	Agent / Tool	Artifact Generated	Enterprise Equivalent
1	RequirementsAgent	Project spec: TypedDict schema, NFR list, architecture layer map, E2A class mapping	ABAP package design + transport structure + interface contract
2	IaCAgent	Terraform: ECS/EKS/Cloud Run, ALB, SQS/DLQ, OpenSearch, Secrets Manager, VPC	SAP system landscape provisioning (DEV → QA → PROD)
3	CodeGenAgent (Observability)	Observability module: structlog JSON, OTel tracing, CloudWatch custom metrics, SLO checker, error budget	CDS analytical views + SAP Cloud ALM integration
4	CICDAgent	GitHub Actions: OIDC auth, RAGAS eval gate, Trivy scan, ECR push, ECS deploy, auto-rollback	SAP CTS+ transport with ATC approval gates
5	CodeGenAgent (NFRs)	NFR module: pybreaker circuit breakers, tenacity retry, FinOps token routing, policy-as-code YAML, LLM cache	ABAP resilience framework + authorisation objects
6	CodeGenAgent (Business Logic)	Agent classes, RAG pipeline, tool microservices, FastAPI routers, LangGraph workflow — built on Phase 1-5 foundation	BIMP classes, determinations, actions — built on tested foundation
7	CodeCriticAgent	Validation report: NFR patterns confirmed present, architecture layers verified, quality score ≥ 0.75	ATC transport check + authorisation review + quality gate

6. Domain-Specific Use Cases

A2C is applicable to any project that needs enterprise-grade generated code. The following use cases demonstrate the breadth of the framework.

Domain	DevRequest	Generated Artifacts	Mandatory NFRs Injected
SAP TM Settlement Engine	FastAPI Microservice, AWS, Python, context: SAP OData integration, high-volume freight orders	Settlement orchestrator, ledger service, audit-AI service, Terraform ECS, GitHub Actions with RAGAS gate	Idempotency (DynamoDB conditional write), double-entry ledger invariant, saga compensation, RAGAS CI/CD gate, FinOps routing
Order-to-Cash Agentic AI	LangGraph Agent, AWS, Python, context: 5-agent orchestration, hybrid RAG, MCP tools	BaseAgent subclasses, BaseRAGPipeline, BaseToolService, Terraform OpenSearch, GitHub Actions with OIDC	All E2A NFRs enforced: idempotency, circuit breakers, groundedness SLO, governance approval, latency SLO
Payment Reconciliation Engine	Spring Boot Microservice, AWS, Java, context: reconcile payments vs ledger, fraud detection	Spring Boot service layers, @Idempotent AOP, Kafka consumer, Terraform EKS, GitHub Actions	@Idempotent annotation, double-entry invariant, Kafka DLQ, structured logging, circuit breaker per downstream
Developer Platform Itself	A2C Platform, Any Cloud, Python, context: generate enterprise-grade code from requirements	SDLCAssistantAgent, CodeCriticAgent, IaCAgent, CICDAgent, CodeGenTool, LangGraph SDLCWorkflow	All above: A2C is its own reference implementation

7. A2C Tool Services

Each code generation capability is a BaseToolService subclass. This ensures idempotency, latency SLO, governance approval, and structured logging are enforced for every file write and validation call — not just the LLM calls.

CodeGenTool — writes generated code to disk / repository

```
# BaseToolService subclass: write generated code files
# Idempotency: same project_name + file_path = overwrite (idempotency=True)
class CodeGenTool(BaseToolService):
    tool_name = 'code_gen'

    def _get_endpoint(self, config=None, **kwargs) -> str:
        return resolve_config('tool_base_url', config) + '/codegen/write'

    def _build_payload(self, payload, config=None, **kwargs) -> dict:
        return {
            'project_name': payload['project_name'],
            'file_path':    payload['file_path'],
            'content':      payload['content'],
            'language':     payload.get('language', 'python'),
            'overwrite':    payload.get('overwrite', True)
        }

    def _validate_input(self, payload, config=None, **kwargs) -> bool:
        return all(k in payload for k in
                   ['project_name','file_path','content'])

# IaCGenTool — validates Terraform before writing
class IaCGenTool(BaseToolService):
    tool_name = 'iac_gen'

    def _validate_input(self, payload, config=None, **kwargs) -> bool:
        # Terraform plan validation before writing IaC files
```

```
        return ('terraform_hcl' in payload and
                len(payload['terraform_hcl']) > 100)
```

Tool	Operation	Idempotency Contract	Validation Rule
CodeGenTool	**WRITE**	project_name + file_path as idempotency key. Same key = overwrite (expected). Different key = new file.	file_path, content, project_name required
IaCGenTool	**WRITE**	project_name + resource_type as idempotency key. Prevents duplicate Terraform resource generation.	terraform_hcl > 100 chars; valid HCL syntax
CICDGenTool	**WRITE**	project_name + workflow_name as idempotency key. Same workflow = overwrite.	workflow_yaml contains OIDC auth, build, deploy stages
TerraformValidateTool	**READ**	No idempotency (tool.idempotency = False). Read-only validation call.	HCL syntax validation via terraform validate

8. Quick Reference

8.1 A2C vs E2A — At a Glance

Aspect	E2A Framework	A2C Framework
What it governs	Agentic AI applications (agents, RAG, tools, workflows)	Developer platform agents that generate enterprise software
Input to run()	AgentState (query, intent, retrieved_docs, response)	DevRequest (project_name, project_type, mandatory_nfrs, target_cloud)
Output of run()	Updated AgentState with response, tokens, trace_id	Updated DevRequest with generated_code, generated_iac, generated_cicd, critic_report
_apply_policy enforces	Tool deny-lists, must-cite rules, approval gates, governance violations	Mandatory NFR injection, approved cloud validation, business dedup key
Quality gate	CriticAgent: RAGAS groundedness ≥ 0.85	CodeCriticAgent: structural NFR pattern check ≥ 0.75 of mandatory patterns
Tool layer	BaseToolService: HTTP API calls to microservices	BaseToolService: file write, IaC validate, YAML generate

8.2 DevRequest Resolution Chain

Resolution follows the central E2A_CONFIG registry (Implementation Playbook Section 5): call-time kwargs > config dict (developer input and _apply_policy injection) > environment variable > registered default, always through resolve_config(). A2C-specific keys are added with register_config_key(); no A2C class calls os.getenv().

```
# Developer provides:
state = {'project_name':'SettlementEngine','project_type':'FastAPI Microservice',
         'mandatory_nfrs':['Observability'],'target_cloud':'aws'}

# _apply_policy injects:
state['mandatory_nfrs'] += ['Idempotency','CircuitBreaker','RetryWithBackoff',
                            'HealthCheck','GracefulShutdown']  # policy-driven
state['generation_key']  = 'SettlementEngine:FastAPI Microservice:aws'

# BaseAgent.__ensure_idempotency adds:
```

```
state['request_id'] = 'uuid4-...'  # technical dedup, prevents re-generation

# BaseAgent.run() adds:
state['trace_id']   = 'uuid4-...'  # observability correlation ID
```

8.3 Implementation Checklist

New SDLCAgent subclass:

- Define agent_name class attribute
- Add to config['approved_agents']
- Override _validate_state: check required DevRequest fields for this agent's domain
- Override _apply_policy: inject domain-specific mandatory NFRs and validate approved_clouds
- Override _build_messages: construct generation prompt encoding E2A patterns as hard requirements
- Override _evaluate_output: structural keyword check OR delegate to CodeCriticAgent
- Override _fallback: return link to E2A template or regeneration instruction
- DO NOT override run() — all NFR enforcement inherited from BaseAgent
- Set max_latency: 30.0 (code generation takes longer than agent calls)
- Set max_tokens: 16000-32000 (code generation requires large token budgets)